



Experiment – 1

Name: Renulasya reddy

UID: 24BAI70210

Branch: CSE – AIML

Section/Group: 24AIT-NTTP-2[A]

Semester: 5th

Date of Performance: 10 /07/2026

Subject: Full Stuck - II

Subject Code: 24CSP-337

EXPERIMENT – 1.1

1. AIM: To design and develop a dynamic post composer interface supporting multiple platforms with constraint validation.

2. OBJECTIVE:

1. To understand multi-platform content handling.
2. To implement real-time validation mechanisms.
3. To design responsive and user-friendly UI components.

3. SOFTWARE REQUIREMENTS:

1. Windows/Linux/macOS
2. Node.js (LTS Version)
3. NPM (Node Package Manager)
4. Visual Studio Code
5. Google Chrome / Microsoft Edge
6. React.js
7. Command Prompt / Terminal

4. THEORY

React is an open-source JavaScript library developed by Meta (Facebook) for building fast, dynamic, and interactive user interfaces. It follows a component-based architecture, where the user interface is divided into reusable components. This approach improves code reusability, readability, and maintainability.

In modern social media applications, users often create content for multiple platforms such as Twitter, LinkedIn, and Instagram. Each platform has different character limits and posting rules. Therefore, a Post Composer is designed to help users create content while validating platform-specific constraints before publishing. Real-time validation improves user experience by immediately informing users if the entered content exceeds the allowed limit.

The Post Composer developed in this experiment is implemented using React Functional Components and React Hooks. The `useState()` hook is used to store and manage dynamic data such as the selected platform and the content entered by the user. Whenever the state changes, React automatically updates the user interface without reloading the entire webpage.

User interactions such as typing in the textarea or selecting a platform are handled using event handling (onChange). The application also uses controlled components, where the value of form elements is managed through React state. This ensures synchronization between the user interface and the application's data.

The application dynamically calculates the number of characters entered using the length property and compares it with the predefined character limit of the selected platform. Based on this comparison, conditional rendering is used to display appropriate validation messages such as "Ready to Post" or "Character Limit Exceeded."

Overall, this experiment demonstrates the practical implementation of React concepts including components, JSX, state management, event handling, controlled components,

5. IMPLEMENTATION/PROCEDURE

1. Install Node.js and verify the installation using `node -v` and `npm -v`.
2. Create a React project using `npx create-react-app post-composer`.
3. Open the project in Visual Studio Code and run it using `npm start`.
4. Design the Post Composer interface with a heading, platform dropdown, and textarea.
5. Implement React State (`useState`) to manage the selected platform and post content.
6. Add a live character counter using the length property.
7. Define platform-specific character limits (Twitter, LinkedIn, and Instagram).
8. Implement real-time validation to display appropriate messages based on the selected platform's character limit.
9. Test the application by entering different post lengths and switching between platforms.
10. Verify that the application displays the correct character count and validation message for each platform.

6. CODE

```
import { useState } from "react";

function App() {
  const [platform, setPlatform] = useState("Twitter");
  const [post, setPost] = useState("");

  const limits = {
    Twitter: 280,
    LinkedIn: 3000,
    Instagram: 2200,
  };

  const limit = limits[platform];
  const isExceeded = post.length > limit;

  return (
```

```
<div style={{ padding: "20px" }}>
  <h1>Post Composer</h1>

  <label><b>Select Platform:</b></label>

  <br /><br />

  <select
    value={platform}
    onChange={(e) => setPlatform(e.target.value)}
  >
    <option>Twitter</option>
    <option>LinkedIn</option>
    <option>Instagram</option>
  </select>

  <br /><br />

  <label><b>Write Your Post:</b></label>

  <br /><br />

  <textarea
    rows="6"
    cols="60"
    placeholder="Type your post here..."
    value={post}
    onChange={(e) => setPost(e.target.value)}
  ></textarea>

  <br /><br />

  <p>
    Characters: {post.length} / {limit}
  </p>
```

```

{isExceeded ? (
  <p style={{ color: "red", fontWeight: "bold" }}>
    ✖ Character limit exceeded!
  </p>
) : (
  <p style={{ color: "green", fontWeight: "bold" }}>
    ✔ Ready to post
  </p>
)}

<br />

<button>Save Draft</button>
</div>

);
}

```

```
export default App;
```

7. OUTPUT

The Social Media Post Composer is a user-friendly interface that allows users to create and manage social media posts according to the selected platform's constraints. Users can select a platform such as Twitter, enter their post content in the text area, and view the real-time character count, which is compared with the platform's maximum character limit. The system displays a status message such as “Ready to Post” when the content is within the allowed limit and provides a “Save Draft” button to save the written content for future use. This interface demonstrates the use of platform-specific constraints, interactive form elements, real-time validation, and dynamic user feedback in a React-based application.

← ↻ ⓘ about:blank

Post Composer

Select Platform:

Instagram ▾

Write Your Post:

I am renulasya - 24BAI70210

Characters: 28 / 2200

✓ Ready to post

Save Draft

Fig 1.1: Post Composer UI with Platform Constraints

8. LEARNING OUTCOME

- Understand the basic concepts and purpose of a social media post composer interface.
- Design and develop a user-friendly post composer UI using React.js.
- Implement platform selection features for platforms such as Twitter, LinkedIn, and Instagram.
- Apply platform-specific character limits to ensure content complies with each platform's constraints.
- Use React state management to handle user input and dynamic UI updates.
- Display real-time character counts and warning messages when the content exceeds the allowed limit.
- Improve user experience through responsive and interactive interface components.
- Develop practical skills in building reusable, constraint-based frontend applications.